

Windows GDI 画笔、画刷与设备环境

以下内容以 Python 的 `pywin32` 为例说明。`pywin32` 对应的底层接口仍然是 Windows GDI (Graphics Device Interface) API, 因此示例中的函数名与 C/C++ Win32 编程中的函数名基本一致。

1、如何调用、创建画笔和画刷

1.1 画笔和画刷的作用

- **画笔 (Pen, `HPEN`)** : 用于绘制直线、矩形边框、椭圆边框等轮廓, 可以设置线型、宽度和颜色。
- **画刷 (Brush, `HBRUSH`)** : 用于填充矩形、椭圆、多边形等封闭图形的内部, 可以设置纯色、阴影或图案。

绘图时, 必须先把画笔或画刷选入设备环境 (DC) 中, 绘图结束后恢复原来的对象, 再删除自己创建的 GDI 对象。

1.2 创建画笔

Win32 API 中常用的创建函数是:

```
HPEN hPen = CreatePen(PS_SOLID, 2, RGB(255, 0, 0));
```

参数含义如下:

- `PS_SOLID`: 实线, 也可以使用 `PS_DASH`、`PS_DOT` 等线型;
- `2`: 画笔宽度;
- `RGB(255, 0, 0)`: 画笔颜色, 这里为红色。

在 Python 中使用 `pywin32`:

```
import win32api
import win32con
import win32gui

# 创建一支 2 像素宽的红色实线画笔
red = win32api.RGB(255, 0, 0)
h_pen = win32gui.CreatePen(win32con.PS_SOLID, 2, red)
```

也可以直接调用系统预置画笔, 例如:

```
h_pen = win32gui.GetStockObject(win32con.BLACK_PEN)
```

常见的预置画笔包括 `BLACK_PEN`、`WHITE_PEN` 和 `NULL_PEN`。`NULL_PEN` 表示不绘制边框。

1.3 创建画刷

Win32 API 中创建纯色画刷的函数是：

```
HBRUSH hBrush = CreateSolidBrush(RGB(0, 255, 0));
```

在 Python 中：

```
# 创建绿色纯色画刷
green = win32api.RGB(0, 255, 0)
h_brush = win32gui.CreateSolidBrush(green)
```

还可以创建阴影画刷或图案画刷，例如底层 API 中的 `CreateHatchBrush` 和 `CreatePatternBrush`。对于系统预置画刷，可以使用：

```
h_brush = win32gui.GetStockObject(win32con.WHITE_BRUSH)
```

如果不希望填充图形内部，可以使用 `NULL_BRUSH`。

1.4 将画笔和画刷选入设备环境

假设已经获得了设备环境句柄 `hdc`，可以使用 `SelectObject`：

```
old_pen = win32gui.SelectObject(hdc, h_pen)
old_brush = win32gui.SelectObject(hdc, h_brush)

try:
    # 画一个红色边框、绿色填充的矩形
    win32gui.Rectangle(hdc, 50, 50, 250, 150)
finally:
    # 恢复原来的画笔和画刷
    win32gui.SelectObject(hdc, old_pen)
    win32gui.SelectObject(hdc, old_brush)

    # 只删除自己创建的对象；系统预置对象不要删除
    win32gui.DeleteObject(h_pen)
    win32gui.DeleteObject(h_brush)
```

其中：

- `SelectObject` 的返回值是原来选入 DC 的对象；
- 恢复原对象后，才能安全删除新创建的画笔或画刷；
- `GetStockObject` 得到的系统对象不应调用 `DeleteObject` 删除；
- 如果对象仍在 DC 中就被删除，可能导致绘图错误或 GDI 资源泄漏。

1.5 使用系统颜色画刷

如果希望画刷颜色跟随 Windows 主题，可以调用 `GetSysColorBrush`：

```
h_brush = win32gui.GetSysColorBrush(win32con.COLOR_WINDOW)
```

系统颜色画刷由 Windows 管理，通常不需要由程序调用 `DeleteObject` 释放。

2、如何调用设备画图环境

Windows GDI 中的设备画图环境称为 **设备环境 (Device Context, DC)**，句柄类型为 `HDC`。它保存当前的画笔、画刷、字体、颜色、映射模式和裁剪区域等绘图状态。

2.1 在窗口客户区获取 DC： `GetDC`

在 Python 中，可以通过窗口句柄 `hwnd` 获取窗口客户区的设备环境：

```
hdc = win32gui.GetDC(hwnd)
try:
    win32gui.MoveToEx(hdc, 20, 20)
    win32gui.LineTo(hdc, 200, 100)
finally:
    win32gui.ReleaseDC(hwnd, hdc)
```

对应的 C/C++ 写法是：

```
HDC hdc = GetDC(hwnd);
MoveToEx(hdc, 20, 20, NULL);
LineTo(hdc, 200, 100);
ReleaseDC(hwnd, hdc);
```

`GetDC` 得到的是客户区 DC，坐标原点通常位于客户区左上角，单位通常为像素。使用完毕后必须调用 `ReleaseDC`。

如果需要包括标题栏和边框在内的整个窗口，可以使用 `GetWindowDC`，释放时同样使用 `ReleaseDC`：

```
hdc = win32gui.GetWindowDC(hwnd)
try:
    # 在整个窗口区域进行绘图
    pass
finally:
    win32gui.ReleaseDC(hwnd, hdc)
```

2.2 在 `WM_PAINT` 中获取 DC： `BeginPaint`

窗口需要重绘时，Windows 会发送 `WM_PAINT` 消息。处理该消息时，应使用 `BeginPaint` 和 `EndPaint`：

```
import win32con
```

```

import win32gui

def wnd_proc(hwnd, message, wparam, lparam):
    if message == win32con.WM_PAINT:
        hdc, paint_struct = win32gui.BeginPaint(hwnd)
        try:
            win32gui.MoveToEx(hdc, 20, 20)
            win32gui.LineTo(hdc, 200, 100)
            win32gui.TextOut(hdc, 20, 120, "Hello GDI")
        finally:
            win32gui.EndPaint(hwnd, paint_struct)
        return 0

    if message == win32con.WM_DESTROY:
        win32gui.PostQuitMessage(0)
        return 0

    return win32gui.DefWindowProc(hwnd, message, wparam, lparam)

```

对应的 C/C++ 形式为：

```

case WM_PAINT:
{
    PAINTSTRUCT ps;
    HDC hdc = BeginPaint(hwnd, &ps);
    MoveToEx(hdc, 20, 20, NULL);
    LineTo(hdc, 200, 100);
    EndPaint(hwnd, &ps);
    return 0;
}

```

`BeginPaint` 除了返回 DC，还会取得当前需要重绘的无效区域，并准备绘制状态；`EndPaint` 会结束本次绘制并通知系统重绘已经完成。因此，处理 `WM_PAINT` 时应优先使用 `BeginPaint`，而不是直接使用 `GetDC`。

2.3 创建内存 DC 进行离屏绘图

为了减少屏幕闪烁，可以创建与窗口 DC 兼容的内存设备环境，先在内存中完成绘制，最后一次性复制到窗口：

```

width, height = 400, 300
window_dc = win32gui.GetDC(hwnd)
memory_dc = win32gui.CreateCompatibleDC(window_dc)
bitmap = win32gui.CreateCompatibleBitmap(window_dc, width, height)
old_bitmap = win32gui.SelectObject(memory_dc, bitmap)

try:
    # 在 memory_dc 上绘制
    win32gui.Rectangle(memory_dc, 0, 0, width, height)

    # 将内存 DC 的内容复制到窗口 DC

```

```
win32gui.BitBlt(  
    window_dc, 0, 0, width, height,  
    memory_dc, 0, 0, win32con.SRCCOPY  
)  
finally:  
    win32gui.SelectObject(memory_dc, old_bitmap)  
    win32gui.DeleteObject(bitmap)  
    win32gui.DeleteDC(memory_dc)  
    win32gui.ReleaseDC(hwnd, window_dc)
```

这里的释放顺序很重要：先把位图恢复为原对象，再删除位图；先删除内存 DC，再释放窗口 DC。

总结

1. 使用 `CreatePen` 创建画笔，使用 `CreateSolidBrush`、`CreateHatchBrush` 等函数创建画刷。
2. 用 `SelectObject` 将画笔和画刷选入 `HDC`，绘制结束后恢复旧对象并释放自己创建的 GDI 对象。
3. 普通窗口绘图可以使用 `GetDC` / `ReleaseDC`。
4. 在 `WM_PAINT` 消息中应使用 `BeginPaint` / `EndPaint`。
5. 需要防止闪烁或进行复杂绘图时，可以使用 `CreateCompatibleDC` 创建内存 DC，再使用 `BitBlt` 显示结果。